AI_CONCIERGE / UI_TECH_PROGRESS_ROADMAP.md

AI Concierge Platform

UI/UX Recap, Technical Progress & Implementation Roadmap

Design • Backend • ML/AI • Development • MLOps

Prepared by: Sharanya, M.Tech, PES University
Report type: Mentor Progress Review — Combined Two-Week Overview

Contents

Part 1	Last Week Recap — Frontend UI/UX Design	ux/
Part 2	Current Progress — Backend & Documentation	progress/
Part 3	Future Roadmap — Implementation	roadmap/

PART 1 — Last Week Recap

Frontend UI/UX design — condensed for continuity, not a re-presentation.

01 What UI/UX Covers, and Who It's For

`ux/`

- UI is the look of every screen; UX is how it feels to actually use the app end to end.
- Design decisions map directly onto the five user-facing capabilities presented last week: Chat, RAG, Memory, Planner, Recommendations.
- Five target audience segments identified, with particular emphasis on Indian-language speakers.

02 Design Philosophy & Five Principles

`design/`

- Single governing rule: keep it simple.
- Five concrete principles operationalize that rule for every screen — consistency, clarity, minimal cognitive load, progressive disclosure, and forgiving interactions.
- Every layout and interaction decision downstream traces back to these five principles.

03 Application Layout & Navigation

`layout/`

- Three-zone layout: sidebar, main workspace, right-hand utility panel — the 'organized desk' analogy.
- A persistent navigation menu is present across every screen, so orientation never resets between features.

04 Theme, Branding & Responsive Design

`theme/`

- Defined color palette and typography hierarchy, with three theme modes.
- Layout adapts across desktop, tablet and mobile; mobile introduces a floating chat button in place of the full three-zone layout.

05 Accessibility

`a11y/`

- Six accessibility commitments made at design time, not deferred to later.
- Includes keyboard navigation, screen-reader support, color contrast, and adjustable text size.

06 Experience Guidelines

`ux/`

- Chat, Documents, Planner: concrete UX defined, including citations shown for explainability — answers point back to their source in the right-hand utility panel.
- Recommendations & Memory: user stays in control — recommendations can be dismissed or saved, and memories can be viewed, edited, or deleted directly.

07 Screen Overview

`screens/`

- Seven screens, each mapped to a feature already introduced: Chat, Documents, Planner, Recommendations, Memory, Settings, and the persistent navigation shell.
- Future enhancements were also scoped but explicitly marked out of current commitment.

Callback: last week's design and this week's technical documentation are meant to be consistent — the memory-RAG-agent integration points documented this week exist specifically to serve the five features designed last week.

PART 2 — Current Progress

Backend, ML/AI, development and MLOps documentation completed this cycle.

08 Project at a Glance

recap/

- Production-grade AI Concierge combining conversational AI, personalization, long-term memory, RAG, tool usage and agentic workflows.
- Core capabilities: authentication, chat, document intelligence, personalization, recommendations and planning.
- Current milestone: documentation and architecture foundation completed; ready to transition into implementation.

09 What Has Been Completed

recap/

- Product requirements, vision, personas, user stories and functional/non-functional requirements documented.
- System architecture, backend architecture, database, ER model, API, authentication, memory, RAG, security, testing, deployment and observability documented.
- Backend-specific documentation organized under `backend_docs/` and ML/AI documentation under `ml/`.
- Development and MLOps documentation completed.
- Canonical repository/documentation structure finalized.

10 Backend Progress

`backend_docs/`

- Backend architecture and API specifications documented.
- Database entities, relationships and persistence strategy defined.
- Authentication, authorization, validation and security considerations documented.
- Service/repository boundaries and integration points established.
- Memory, RAG, agents and external-service integration points identified.

11 ML / AI Progress

`ml/`

- ML architecture and data-processing considerations documented.
- Embeddings, retrieval, reranking and RAG pipeline documented.
- LLM integration and prompt-engineering considerations documented.
- Memory, personalization and recommendation components documented.
- Evaluation and AI-safety considerations included.

12 Development Foundation

`dev/`

- Repository organization separates documentation from implementation.
- Coding standards, Git workflow, environment setup, local development, deployment and troubleshooting are documented.
- Implementation is planned incrementally so each layer can be tested before the next is introduced.

13 MLOps / LLMOps Foundation

mlops/

- MLOps pipeline and LLMOps lifecycle documented.
- CI/CD, model versioning and prompt versioning documented.
- AI monitoring covers latency, errors, tokens, cost, retrieval, memory, tools and model behavior.
- Evaluation covers correctness, relevance, groundedness, RAG, agents, tools, memory, safety, latency and cost.

14 Technology Stack

stack/

Frontend	React + TypeScript
Backend / API	FastAPI
Database	PostgreSQL
Vector DB	Qdrant
LLM	Gemini / OpenAI-compatible APIs
Deployment	Docker
CI/CD	GitHub Actions
Monitoring	Prometheus + Grafana
Experiment Tracking	MLflow

15 Target Architecture

`architecture/`

Frontend requests pass through the FastAPI API Gateway into an Agent Orchestrator, which coordinates Memory, RAG and Tools before the LLM Service generates the final response. PostgreSQL stores structured application data; Qdrant stores vector representations.

Orchestrator coordinates:

Planned agents: `Intent, Memory, Retrieval, Tool, Response`. Evaluation Agent is future scope.

16 Current Status

`status/`

- Major architectural decisions and component boundaries are documented.
- Development and MLOps workflows are defined.
- Repository structure supports incremental implementation.
- Project has moved from documentation readiness to implementation readiness.

Immediate objective: first working end-to-end application flow.

PART 3 — Future Roadmap

Nine implementation phases, the six-month plan, MVP scope, and risks.

17 Implementation Phases (1 of 2)

roadmap/

Phase 1: Foundation

1. Initialize repository and source-code structure.
2. Set up backend/frontend environments and configuration.
3. Create .env.example and initialize Git baseline.
4. Create backend health endpoint.
5. Initialize frontend.
6. Connect frontend → backend through the first API call.

Goal: a running skeleton with frontend and backend able to talk to each other.

Phase 2: Database

1. Configure PostgreSQL.
2. Implement models from the finalized Database Design and ER Diagram.
3. Set up migrations.
4. Implement repository/data-access and service layers.
5. Add CRUD operations and database tests.
6. Verify end-to-end persistence through backend APIs.

Goal: data reliably flows from API calls into PostgreSQL and back.

Phase 3: Authentication & APIs

1. Registration and login.
2. Password hashing and JWT authentication.
3. Authorization and validation.
4. User/profile APIs.
5. Conversation and message APIs.
6. Error handling and API tests.
7. Secure secrets and sensitive configuration.

Goal: a secure, authenticated foundation for every feature built afterward.

18 Implementation Phases (2 of 2)

roadmap/

Phase 4: First Working Chat

1. Build the basic chat UI.
2. Connect chat UI to backend.
3. Persist conversations.
4. Integrate the selected LLM through a controlled service interface.
5. Identify prompt versions.

Goal: user message → backend → LLM → response → stored conversation.

Phase 5: Memory

1. Implement short-term context.
2. Implement persistent memory.
3. Define memory creation/update rules.
4. Implement retrieval and relevance filtering.
5. Connect relevant memory to prompt construction.
6. Evaluate whether memory improves personalization.

Goal: the assistant remembers and uses relevant context across sessions.

Phase 6: RAG

1. Document upload and parsing.
2. Chunk documents.
3. Generate embeddings.
4. Store vectors in Qdrant.
5. Implement retrieval and optional reranking.
6. Inject context into the LLM.
7. Return grounded answers and evaluate retrieval/groundedness.

Goal: answers are grounded in uploaded documents, not just model memory.

Phase 7: Agents & Tools

1. Define controlled tool interfaces.
2. Validate and authorize tool calls.
3. Introduce orchestration for genuinely multi-step tasks.
4. Add iteration limits and timeouts.
5. Handle tool/agent errors.
6. Evaluate tool selection, arguments and task completion.

Goal: multi-step tasks execute reliably, safely, and within bounds.

Phase 8: Testing & Evaluation

1. Unit, integration, API and end-to-end testing.
2. RAG retrieval and groundedness evaluation.
3. Memory relevance evaluation.
4. Agent/tool evaluation.
5. Regression dataset and baseline comparisons.
6. Quality gates before major releases.

Goal: quality is measured continuously, not assumed.

Phase 9: MLOps & Production

1. Dockerize services.
2. Implement CI/CD.
3. Track model and prompt versions.
4. Integrate experiment/evaluation tracking.
5. Add structured logs, metrics and traces.
6. Monitor latency, errors, token usage, cost and AI quality.
7. Deploy staging before production.

Goal: the system is observable, versioned, and safely deployable.

19 Six-Month Roadmap

timeline/

Weeks 1–4	Backend, authentication and chat foundation
Weeks 5–8	Memory system
Weeks 9–12	RAG pipeline
Weeks 13–16	Agent framework
Weeks 17–20	Personalization layer
Weeks 21–24	Docker, CI/CD, monitoring and deployment

20 MVP Definition of Done

mvp/

- Authenticated user flow works.
- Conversation creation, continuation and persistence work.
- Basic LLM-powered chat works.
- Document upload and document Q&A; work.
- RAG retrieves relevant information and produces grounded responses.
- Basic persistent memory enables personalization.
- Core workflows are tested and observable.
- Application can be containerized and deployed.

21 Risks & Mitigation

`risks/`

LLM/API cost	Caching, usage monitoring and model selection.
Hallucination	RAG grounding, evaluation and fallback behavior.
Poor retrieval	Retrieval evaluation, chunking and reranking improvements.
Agent loops	Iteration limits, timeouts and tool controls.
Security / privacy	Authentication, validation, secrets management and careful telemetry.
Scope growth	Phased MVP and milestone-based implementation.

22 Mentor Discussion Points

`review/`

- Validate the implementation sequence.
- Review backend vs ML component boundaries.
- Confirm MVP scope before significant coding.
- Prioritize database/authentication/chat milestones.
- Review memory → RAG → agents integration order.
- Identify architectural risks and measurable acceptance criteria.

23 Immediate Next Milestone

`milestone/`

1. Move from documentation to implementation.
2. Build project skeleton and connect FastAPI to PostgreSQL.
3. Implement first database models and migration.
4. Create authentication foundation.
5. Build first frontend → backend flow.
6. Then add LLM integration and progressively introduce memory, RAG and agents.

24 Closing

`closing`

Design	Documentation	Architecture	Implementation	Deployment

Last week's design work and this week's technical documentation together give the project a structured foundation for disciplined implementation.

Immediate objective: a small, working, testable end-to-end prototype — not the complete AI system at once.

— End of Report —